Copyright © 2023. Bentham Science Publishers. All rights reserved.

Exception Handling

Krishna Kumar Mohbey[1], Malika Acharya[1]

[1] Computer Science, Central University of Rajasthan, India

Abstract

In this chapter, we introduce the concept of exception handling in Python. They are used to specify the alternate sequence of actions the program needs to jump to at the occurrence of the event. For example, if we want to print several pages from the printer and somewhere in the middle of the job the paper gets stuck in the printer. In such as situation we would want to jump to the function that aborts the printing and handles this situation by instant shut down of the printer. In such events comes the exception handling. When the program jumps to the exception handler part the current sequence of commands is abandoned and the commands given to the exception handler are executed. After the exception is tackled the programming returns to the point where the marker left.

Keywords: Error handling, Event notification.

Mohbey, Krishna Kumar, and Malika Acharya. Basics of Python Programming: a Quick Guide for Beginners, Bentham Science Publishers, 2023. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=31025226.
Created from univ-people-ebooks on 2026-10-22 04:40:18.

Copyright © 2023. Bentham Science Publishers. All rights reserved.

INTRODUCTION

Error handling makes your code more robust by shielding it from the kinds of mistakes that may result in an abrupt shutdown of your application. On the other hand, Python exceptions can be handled in contrast to errors. Errors can be syntax errors, and although various exceptions might happen during execution, they aren't always unusable. An error could be a syntax mistake. An application that is reasonable and well-designed should avoid the critical issues indicated by an Error.

In contrast, an application that is suitable and well-designed should try to capture the conditions that are indicated by an Exception. Programmers should avoid handling errors wherever possible since they are a form of uncontrolled exception that cannot be recovered from. An example of this type of error is the **ZeroDivisionError**. Think about what would happen if you produced code that was later used in production but still ended because of an error. Because the customer would be dissatisfied, handling the exception in advance and eliminating uncertainty is preferable. There are two different kinds of mistakes, the first being syntax errors and the second being exceptions. A syntax error will occur when the parser identifies a grammatical problem in your code. Syntax errors are also commonly referred to as parsing errors. To better understand it, let's look at an example.

Example:

```
1. #Syntax error example
2. x = 1
3. y = 2
4. z = x y
```

Output:

```
File "<string>", line 3
z = x y
^
SyntaxError: invalid syntax
```

Mohbey, Krishna Kumar, and Malika Acharya. Basics of Python Programming: a Quick Guide for Beginners, Bentham Science Publishers, 2023. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=31025226.
Created from univ-people-ebooks on 2026-10-22 04:40:18.

Copyright © 2023. Bentham Science Publishers. All rights reserved.

The arrow in the report shows that the code parser ran into an error when executing the program. The failure may be traced back to the token before the arrow. Because it will output the file's name and the line number where the issue occurred, Python will handle a significant portion of the troubleshooting work for you when attempting to resolve errors of this type.

An error that arises as a result of the execution of a program is referred to as an exception. Exceptions are occurrences that do not obey a general rule, which is how non-programmers understand the term. When a statement or expression is executed, an error through the syntax may occur. Python's exceptions are faults that may be seen during the execution of the program but are not always catastrophic. An exception object is created whenever a Python program generates a runtime error. The program will be terminated unexpectedly and without warning if the code does not expressly handle the exception. In most cases, programs will disregard exceptions, which will lead to error messages such as the following:

Example:

```
1. # Example of type error
2. name= 'David'
3. age=45
4. age+name
```

Output:

```
Traceback (most recent call last):
  File "<string>", line 3, in <module>
TypeError: unsupported operand type(s) for +: 'int' and 'str'
```

Another example of division by zero exception can be seen in the below example.

Example:

```
1. #Example of division by zero exception
2. 10 * (15/0)
```

Output:

```
Traceback (most recent call last):
  File "<string>", line 1, in <module>
ZeroDivisionError: division by zero
```

Mohbey, Krishna Kumar, and Malika Acharya. Basics of Python Programming: a Quick Guide for Beginners, Bentham Science Publishers, 2023. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=31025226.
Created from univ-people-ebooks on 2026-10-22 04:40:18.

Copyright © 2023. Bentham Science Publishers. All rights reserved.

Python exceptions can take several forms, indicated right next to the message they produce. For example, the types of exceptions that have just been discussed are **TypeError** and **ZeroDivisionError**. Both error messages include, the kind of exception, and the name of the Python built-in exception that was encountered. The remaining half of the error line comprises information about what led to the mistake; the specifics of this information are determined by the type of exception that was thrown.

HANDLING EXCEPTIONS

Python's approach to managing exceptions is similar to Java's. The code that might result in the throwing of an exception is contained within a **try block**. In Java, exceptions are handled using catch clauses, but in Python, exceptions are handled by sentences inserted using the **except** keyword. Personalized deviations from the norm can be accommodated if necessary. By utilizing the “raise” command, it is possible to coerce the occurrence of an exception. You may safeguard your application by enclosing any potentially malicious code that can cause an exception in a try: block. This will prevent the exception from being thrown. Place an except declaration after the try: block, then immediately after that, a block of code that fixes the problem in the most elegant way possible should follow it. The following is an example of the syntax for **try...except...else** blocks:

Syntax:

```
try:
        # Staement that may generage exception
 except Exception_1:
        #If Exception_1 occurs, this block will be executed.
except Exception_2:
        # If Exception_2 occurs, this block will be executed.
except Exception_3:
        # If Exception_3 occurs, this block will be executed.
else:
        # If No exception occurs, this block will be executed.
```

The main components of exception handling are shown in Table 1.

Mohbey, Krishna Kumar, and Malika Acharya. Basics of Python Programming: a Quick Guide for Beginners, Bentham Science Publishers, 2023. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=31025226.
Created from univ-people-ebooks on 2026-10-22 04:40:18.

Copyright © 2023. Bentham Science Publishers. All rights reserved.

Table 1 **Exception handling keywords.**

try	It will execute the code block in which you expect an error.
except	It catches the exception, which occurs in a try block.
else	This block of code will be executed if there are no exceptions.
finally	This block of code will always be executed, regardless of whether there is an exception.

Below is an example of a ValueError exception.

Example:

```
1. #Example of ValueError exception
2. try:
3.     myvar = input ("Enter any Number here: ")
4.     myvar = int(myvar)
5.     print(myvar)
6. except ValueError:
7.         print("Exception occurs")
8. else:
9.     print('No exception occurred')
```

Output:

```
------------------------------Run program with following input------------------------------
Enter any Number here: 10
10
No exception occurred
------------------------------Run program with following input------------------------------
Enter any Number here: Python
Exception occurs
```

EXCEPT BLOCK

A single except statement can be used with a try block. This block can catch all types of exceptions. In this type of exception handling, the programmer cannot find the actual reason for the problems.

Mohbey, Krishna Kumar, and Malika Acharya. Basics of Python Programming: a Quick Guide for Beginners, Bentham Science Publishers, 2023. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=31025226.
Created from univ-people-ebooks on 2026-10-22 04:40:18.

Copyright © 2023. Bentham Science Publishers. All rights reserved.

Example:

```
try:
    x = 15
    y = 0
    ans = x/y
    print(ans)
except:
    print("Error: Division by 0 not possible.")
```

Output:

```
Error: Denominator cannot be 0.
```

One **try** block can have many exception blocks attached to it at the same time. Nevertheless, only one of the exemption provisions will be put into effect. Let's look at an example written in Python that uses numerous except blocks in conjunction with a single try block. When the interpreter encounters an exception, it investigates the try block's associated except blocks first. It is possible to define the categories of exceptions that these except blocks will handle. When the interpreter comes across a comparable exception, it will execute unless blocked.

Example:

```
try:
    x1 = int(input("First number:"))
    x2 = int(input("Second number:"))
    sum = x1 + x2
    div =  x1 / x2
    print(sum)
    print(div)
except ZeroDivisionError:
    print("Division by zero")
except ValueError:
    print("Error: Enter an integer only")
```

Output:

```
First number:45
Second number:f
Error: Enter an integer only
```

We can also handle multiple exceptions with a single except block using parentheses to do this. The interpreter will return a syntax error if this is not done.

Mohbey, Krishna Kumar, and Malika Acharya. Basics of Python Programming: a Quick Guide for Beginners, Bentham Science Publishers, 2023. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=31025226.
Created from univ-people-ebooks on 2026-10-22 04:40:18.

Copyright © 2023. Bentham Science Publishers. All rights reserved.

Example:

```
def test(x):
  try:
    x1 = int(x)
    if x1 > 60:
      print("Valid age")
  except TypeError:
    print("x is invalid..")
  except ValueError:
    print("x can only be integer")
  except:
    print("Unexpected error")

----Run the program with following set of outputs---------
x = [3.5,4]
test(x)

-----Run the program with following set of outputs--------
x= 85
test(x)
```

Output:

```
x is invalid...


Valid age
```

ELSE AND FINALLY, KEYWORD

Python's try and except blocks, together with the keywords else and finally, can be used in conjunction with one another. If an error occurs in the try block while the program is being performed, the except block will be carried out instead. However, if the try block does not produce any exceptions, the code in the else block will be carried out. In the end, another statement is utilized in the exception-handling process. Regardless of whether an exception is thrown, the final block's contents are always carried out in their entirety. As a consequence, the error-free try block advances directly to the finally block without running the except clause. It then continues to execute the remaining code.

Syntax:

Mohbey, Krishna Kumar, and Malika Acharya. Basics of Python Programming: a Quick Guide for Beginners, Bentham Science Publishers, 2023. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=31025226.
Created from univ-people-ebooks on 2026-10-22 04:40:18.

Copyright © 2023. Bentham Science Publishers. All rights reserved.


```
try:
   #Statements
except:
   # Statements, if an exception occurs
else:
   #Statements, if no exception occurs
finally:
   #This code will always run
```

Example:

```
1.  def test_except():
2.    try:
3.      a= int(input("Enter input 1:"))
4.      b= int(input("Enter input 2:"))
5.      add= a - b
6.    except TypeError:
7.      print("x is invalid..")
8.    except ValueError:
9.      print("x can only be integer")
10.   else:
11.     print(add)
12.   finally:
13.     print("exception tested")
14. test_except()
```

Output:

```
Enter input 1:4
Enter input 2:5.05
x can only be integer
exception tested
```

RAISING EXCEPTIONS

Python also has the raise keyword, which may be used for managing exceptions in various situations. It causes an explicit exception to be thrown as a consequence. Errors that are built into the system will be raised automatically. However, when executing the code, you can compel either a built-in or a custom exception. The raise statement can be used in the following manner: its syntax is detailed below:

Mohbey, Krishna Kumar, and Malika Acharya. Basics of Python Programming: a Quick Guide for Beginners, Bentham Science Publishers, 2023. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=31025226.
Created from univ-people-ebooks on 2026-10-22 04:40:18.

Copyright © 2023. Bentham Science Publishers. All rights reserved.

Syntax:

```
raise Exception_class (<value>)
```

Example:

```
1. details =  {'Languages':'Python', 'code':'C70458','enrolled': 75}
2. key= input("Search:")
3. try:
4.   print(f'{key} is {details[key]}')
5.   if key not in details:
6.     raise KeyError()
7. except  KeyError:
8.     print("Key used is invalid")
```

Output:

```
Search:name
Key used is invalid
Search:code
code is C70458
```

The below code is also an example of raising an exception. Here ArithmeticError class is used with the raise keyword.

Example:

```
1.   def airth_err(x,y):
2.       try:
3.           print(x/y)
4.       except ZeroDivisionError as e:
5.           print("work complet e)
6.       else:
7.           print("No error")
8.
9.
10.  airth_err(10,5)
11.  airth_err(50,0)
```

Output:

```
2.0
No error
division by zero
```

Mohbey, Krishna Kumar, and Malika Acharya. Basics of Python Programming: a Quick Guide for Beginners, Bentham Science Publishers, 2023. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=31025226.
Created from univ-people-ebooks on 2026-10-22 04:40:18.

Copyright © 2023. Bentham Science Publishers. All rights reserved.

Built-in Exceptions

When errors occur in Python, several built-in exceptions are raised. The ***local()*** built-in functions can be used to display these built-in exceptions as follows:

Example:

```
1.    print (dir(locals()['__builtins__']))
```

Output:

```
['ArithmeticError', 'AssertionError', 'AttributeError', 'BaseException', 'BlockingIOError',
'BrokenPipeError', 'BufferError', 'BytesWarning', 'ChildProcessError',
'ConnectionAbortedError', 'ConnectionError', 'ConnectionRefusedError',
'ConnectionResetError', 'DeprecationWarning', 'EOFError', 'Ellipsis', 'EnvironmentError',
'Exception', 'False', 'FileExistsError', 'FileNotFoundError', 'FloatingPointError',
'FutureWarning', 'GeneratorExit', 'IOError', 'ImportError', 'ImportWarning', 'IndentationError',
'IndexError', 'InterruptedError', 'IsADirectoryError', 'KeyError', 'KeyboardInterrupt',
'LookupError', 'MemoryError', 'ModuleNotFoundError', 'NameError', 'None',
'NotADirectoryError', 'NotImplemented', 'NotImplementedError', 'OSError', 'OverflowError',
'PendingDeprecationWarning', 'PermissionError', 'ProcessLookupError', 'RecursionError',
'ReferenceError', 'ResourceWarning', 'RuntimeError', 'RuntimeWarning', 'StopAsyncIteration',
'StopIteration', 'SyntaxError', 'SyntaxWarning', 'SystemError', 'SystemExit', 'TabError',
'TimeoutError', 'True', 'TypeError', 'UnboundLocalError', 'UnicodeDecodeError',
'UnicodeEncodeError', 'UnicodeError', 'UnicodeTranslateError', 'UnicodeWarning',
'UserWarning', 'ValueError', 'Warning', 'WindowsError', 'ZeroDivisionError',
'__IPYTHON__', '__build_class__', '__debug__', '__doc__', '__import__', '__loader__',
'__name__', '__package__', '__spec__', 'abs', 'all', 'any', 'ascii', 'bin', 'bool', 'breakpoint',
'bytearray', 'bytes', 'callable', 'chr', 'classmethod', 'compile', 'complex', 'copyright', 'credits',
'delattr', 'dict', 'dir', 'display', 'divmod', 'enumerate', 'eval', 'exec', 'execfile', 'filter', 'float',
'format', 'frozenset', 'get_ipython', 'getattr', 'globals', 'hasattr', 'hash', 'help', 'hex', 'id', 'input', 'int',
'isinstance', 'issubclass', 'iter', 'len', 'license', 'list', 'locals', 'map', 'max', 'memoryview', 'min',
'next', 'object', 'oct', 'open', 'ord', 'pow', 'print', 'property', 'range', 'repr', 'reversed', 'round',
'runfile', 'set', 'setattr', 'slice', 'sorted', 'staticmethod', 'str', 'sum', 'super', 'tuple', 'type', 'vars', 'zip']
```

Some important built-in exceptions are described in below Table 2:

Table 2 **Built-in exceptions.**

Name	Condition
AssertionError	when an assertion fails.
AttributeError	when attribute assignment or reference fails
EOFError	when an end-of file is witnessed by input() function

Mohbey, Krishna Kumar, and Malika Acharya. Basics of Python Programming: a Quick Guide for Beginners, Bentham Science Publishers, 2023. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=31025226.
Created from univ-people-ebooks on 2026-10-22 04:40:18.

Copyright © 2023. Bentham Science Publishers. All rights reserved.

FloatingPointError	when there are floating point errors
GeneratorExit	on the call of close() operation of generator
ImportError	when the module imported is not found
IndexError	At the out-of-bound index
IndentationError	Raised at an erroneous indentation
KeyError	When the key is not present in the dictionary
KeyboardInterrupt	Raised at the interruption of the execution by the user
MemoryError	Raised when the action is out of memory
NameError	Raised when the named variable is not present in the local or global scope
NotImplementedError	Raised by abstract methods
OSError	Raised when there occur OS-related failures
OverflowError	Raised when the result of the operation is too large to be expressed
ReferenceError	At the use of a weak reference proxy to access a garbage collection, this error is raised
RuntimeError	When the error is out of the scope of any other category
StopIteration	When the iterator reaches the end and there are no more items to return, the next() function raises this error
SyntaxError	Raised at the occurrence of syntax mistakes
SystemError	Raised at the occurrence of internal error
SystemExit	Raised by sys.exit() function.
TabError	Raised when there is a mix of erroneous tabs and space.
TypeError	Raised when the function is applied to the wrong object type.
UnboundLocalError	Raised when there is a reference to a local variable that the value has not bounded
UnicodeError	At the occurrence of Unicode-related encoding and decoding
UnicodeEncodeError	When the Unicode-related error occurs during encoding
UnicodeDecodeError	When the Unicode-related error occurs during decoding
UnicodeTranslateError	When a Unicode-related error occurs during translation
ValueError	Raised when the function receives the argument of valid type but the

Mohbey, Krishna Kumar, and Malika Acharya. Basics of Python Programming: a Quick Guide for Beginners, Bentham Science Publishers, 2023. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=31025226.
Created from univ-people-ebooks on 2026-10-22 04:40:18.

Copyright © 2023. Bentham Science Publishers. All rights reserved.

	wrong value
ZeroDivisionError	Raised at the division or modulo with zero

User-defined Exceptions

You may create your own exceptions by creating classes from the built-in exceptions in Python. Programmers can invoke their exceptions by developing a new exception class. Inheritance from the Exception class is required for exceptions, either directly or indirectly. An illustration of a user-defined exception may be seen in the code below:

Example:

```
1.  # define Python user-defined exceptions
2.  class SeniorCitizenException(Exception):
3.      "Raised when age is less than 60"
4.      pass
5.  age = 60
6.  try:
7.      yr_age= int(input("Enter your age: "))
8.      if yr_age >= age:
9.        print("Senior citizen")
10.       print("Eligible")
11.     else:
12.       raise SeniorCitizenException
13. except SeniorCitizenException:
14.     print("Exception occurred: Not eligible")
```

Output:

```
------------------------------Running the program with following inputs------------------------------

Enter your age: 45

Exception occurred: Not eligible

Enter your age: 65

Senior citizen

Eligible
```

Mohbey, Krishna Kumar, and Malika Acharya. Basics of Python Programming: a Quick Guide for Beginners, Bentham Science Publishers, 2023. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=31025226.
Created from univ-people-ebooks on 2026-10-22 04:40:18.

Copyright © 2023. Bentham Science Publishers. All rights reserved.

In the example above, we create a custom exception **SeniorCitizenException** defined in a class derived from the built-in Exception class. When the user's age is less than 60, an exception is generated, and the rest of the statements in the try block are skipped. The user-defined exception *SeniorCitizenException* activates the except block, and the statement inside this block is executed.

CONCLUDING REMARK

In this chapter, we discussed Exception handling in Python. We also discussed the uses of try, except, else, and finally blocks for managing exceptions. In the next chapter, we will discuss the concepts of Modules and packages.

Mohbey, Krishna Kumar, and Malika Acharya. Basics of Python Programming: a Quick Guide for Beginners, Bentham Science Publishers, 2023. ProQuest Ebook Central, http://ebookcentral.proquest.com/lib/univ-people-ebooks/detail.action?docID=31025226.
Created from univ-people-ebooks on 2026-10-22 04:40:18.